TSP Visualizer

Interactive Travelling Salesman Problem Laboratory — a DAA (Design and Analysis of Algorithms) college project that visualizes *why* TSP becomes computationally difficult and how different algorithms deal with that difficulty.

> A small increase in the number of cities can cause an enormous increase in the number of possible tours.

About

This is not just a route finder. It is an **algorithm-learning and complexity-analysis tool**:

- Build a TSP instance visually (drag cities) or by editing the distance matrix.
- Run **Brute Force**, **Held–Karp (DP)**, or **Branch & Bound** step-by-step or automatically.
- Watch routes being explored, DP states being computed, and branches being pruned — live on the graph.
- See real measured statistics (states explored, pruned, execution time) from the actual implementations.
- Explore the complexity explosion: tour counts, growth charts, and a scaling experiment mode.

Features

- **Interactive SVG graph** — draggable cities, complete weighted graph, toggleable distance labels, optional auto Euclidean distances.
- **Editable distance matrix** — symmetric editing with validation (no negatives, no NaN, diagonal stays 0).
- **Three real algorithms**, implemented as steppable runners (one meaningful operation per step):
  - Brute Force — Heap's algorithm over all (n−1)! permutations, start city fixed.
  - Held–Karp — bitmask DP over `dp[mask][i]` states, with route reconstruction.
  - Branch & Bound — DFS with an admissible lower bound (minimum-outgoing-edge relaxation) and a nearest-neighbor initial upper bound.
- **Transport controls** — Run / Pause / Step / Reset + speed slider. Long runs batch steps per animation frame and yield so the browser stays responsive.
- **Algorithm log** — every meaningful event (tour evaluated, new best, DP state computed, branch pruned with its bound reasoning).

- **Live statistics** — cities, best/current distance, states explored, states pruned, pruning rate, execution time, best route.
- **"Why Does TSP Get Hard?"** — tour-explosion table, log-scale growth chart comparing $(n-1)!$ vs $n^2 \cdot 2^n$ vs n^2.
- **Algorithm comparison** — theory (time/space complexity) side-by-side with measured values from your runs.
- **Experiment mode** — runs all three algorithms on deterministic random instances for n = 4…10, with a results table and bar chart. Cancellable, non-blocking.
- **Presets** — 4-city, 6-city, 8-city, symmetric, and random deterministic examples.
- **Reset granularity** — Reset Algorithm / Reset Problem / Reset Everything.

Algorithms

Brute Force

Fix the starting city and enumerate all permutations of the remaining n−1 cities (Heap's algorithm, iterative). Every complete tour is evaluated and compared against the best so far.

- **Time:** $O((n-1)!)$ — (n−1) choices, then (n−2), …, 1.
- **Space:** O(n) for the current route/permutation state.
- The visualization shows exactly why it becomes impractical: every tour is visited, nothing is skipped.

Held–Karp (Dynamic Programming)

Memoizes the subproblem `dp[mask][i]` = cheapest cost to start at city 0, visit exactly the cities in `mask`, and end at city `i`. There are 2^n subsets × up to n endings × up to n transitions.

- **Time:** $O(n^2 \cdot 2^n)$
- **Space:** $O(n \cdot 2^n)$
- Repeated work in brute force becomes a table lookup. The final tour is reconstructed via parent pointers.

Branch & Bound

Depth-first search over the same search tree as brute force, but with **pruning**: for a partial route it computes a *lower bound* (cost so far + cheapest possible way to leave/enter every unvisited city + cheapest return to start). If `lower bound ≥ best known tour`, the whole subtree is discarded — safely, because no completion of that branch can beat the incumbent.

- **Time:** exponential in the **worst case** (pruning does not change the worst-case class).
- **Space:** O(n).

- A nearest-neighbor tour seeds the initial upper bound so pruning starts working early. Children are expanded nearest-first for the same reason.

Tech Stack

- **HTML** — single-page dashboard layout.
- **CSS** — hand-written dark "algorithm laboratory" theme, no frameworks.
- **Vanilla JavaScript** — plain scripts (no modules/bundler) sharing a `window.TSP` namespace, so the app runs by simply opening `index.html` (no build step, no server required).
- **SVG** — for the graph and all charts (precise control over nodes, edges, labels, and highlighting).

How to Run

Option 1 — just open it: double-click `index.html` (works from `file://`, no server needed).

Option 2 — local server (recommended):

```
cd tsp-visualizer
python3 -m http.server 8000
# then open http://localhost:8000
```

Project Architecture

```
tsp-visualizer/
├── index.html          # Dashboard layout: header, graph, control panel, analysis tabs
├── style.css           # Dark lab theme
├── README.md
└── js/
    ├── tsp.js           # Problem model: TSPProblem (cities, symmetric distance
    │                    # matrix, validation, deterministic presets/random)
    ├── algorithms.js    # The three algorithms as steppable runners + headless
    │                    # fast-run helper for Experiment mode
    ├── visualization.js# SVG graph: rendering, dragging, route/state highlighting
    ├── complexity.js   # Safe big-number formatting, log-space complexity math,
    │                    # growth chart + experiment chart drawing
    └── app.js           # UI wiring: transport, run loop, log, stats, matrix
                         # editor, presets, explorer, comparison, experiments
```

Key design idea: each algorithm is a *runner* — `step()` performs exactly one meaningful operation (one tour evaluated / one DP state computed / one search-tree node processed) and returns a log event. The UI run loop batches steps per animation frame according to the speed slider, so stepping, pausing, and animation all share the same code path, and headless

experiment runs reuse the runners too. Every statistic displayed is measured from these implementations — nothing is hardcoded or faked.

Screenshots

Add screenshots here:

- `docs/screenshot-graph.png` — main graph with best route highlighted
- `docs/screenshot-complexity.png` — "Why Does TSP Get Hard?" explorer
- `docs/screenshot-experiment.png` — experiment mode results

Future Improvements

- Asymmetric TSP support (directed distances).
- Alternative lower bounds for B&B (e.g. MST / 1-tree relaxation) with a bound selector.
- Approximation algorithms tab (nearest neighbor, 2-opt, Christofides) to contrast exact vs heuristic.
- Import/export problem instances as JSON.
- Animated search-tree panel for Branch & Bound.
- Touch-friendly matrix editing for small screens.